Estruturas de Dados I

Exercícios: TAD Pilha: Implementação Encadeada e Aplicações

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

8 de outubro de 2026

Exercícios

1. Simule, à mão, o algoritmo de verificação de delimitadores balanceados para cada expressão abaixo. Apresente o conteúdo da pilha após cada delimitador lido e indique se a expressão está balanceada; caso não esteja, indique o motivo da rejeição.
   a. `{a * [b + (c - d)]}`
   b. `[(a + b] * c)`
   c. `((a + b) * [c - d]`
2. Simule, à mão, a avaliação de cada expressão pós-fixa abaixo. Apresente, em uma tabela, o símbolo lido, a ação executada e o conteúdo da pilha após cada passo, e indique o resultado.
   a. `6 2 3 + *`
   b. `8 4 2 / - 3 *`
   c. `8 3 1 - / 5 2 * +`

Exercícios

3. Simule, à mão, a conversão de cada expressão infixa abaixo para a notação pós-fixa. Apresente, em uma tabela, o símbolo lido, a ação executada, o conteúdo da pilha e a saída após cada passo.
   a. `a - b + c * d`
   b. `a * (b + c * d) - e`
   c. `(a + b) / (c - d)`
   d. `a + b * (c - d) / e`
   e. `((a + b) * c - d) * e`

Exercícios

4. Acrescente as seguintes operações à interface do TAD Pilha e implemente-as na implementação encadeada:
   - `exibirPilha()`: imprime os elementos do topo para a base, sem alterar a pilha;
   - `esvaziarPilha()`: remove todos os elementos e libera seus nós, mantendo o registro de controle, de modo que a pilha possa continuar sendo usada.
5. Acrescente a operação `concatenarPilhas()` à interface do TAD Pilha e implemente-a na implementação encadeada. Essa operação recebe duas pilhas e coloca todos os elementos da segunda sobre os da primeira, preservando a ordem da segunda. Ao final, a segunda pilha deve ficar vazia. Por exemplo, se a primeira pilha contém, do topo para a base, 30 20 10, e a segunda contém 3 2 1, a primeira passa a conter 3 2 1 30 20 10.

Exercícios

6. Escreva a função `posicaoErro()` que recebe uma expressão e devolve -1 se os seus delimitadores estiverem balanceados; caso contrário, devolve a posição do delimitador que causou o erro. Se um fechamento não tiver abertura correspondente, ou não combinar com ela, deve ser devolvida a posição desse fechamento. Se sobrarem aberturas ao final, deve ser devolvida a posição da abertura mais recente que não foi fechada. Para isso, a pilha deve guardar as posições das aberturas, e não os próprios caracteres. Exemplos:
   - `(a + [b)]`: devolve 7;
   - `a + b)`: devolve 5;
   - `((a + b)`: devolve 0;
   - `[a * (b + c)]`: devolve -1.

Exercícios

7. Estenda a avaliação de expressões pós-fixas vista em aula para que aceite números inteiros não negativos com mais de um dígito, separados por espaços, e rejeite expressões mal formadas. A função `avaliarPosFixa()` recebe a expressão e o endereço onde o resultado deve ser guardado; devolve `false` se a expressão for mal formada, e `true` caso contrário. A expressão é mal formada quando um operador encontra menos de dois operandos na pilha, quando ocorre uma divisão por zero, ou quando não resta exatamente um valor na pilha ao final. Exemplos:
   - `12 3 4 * +`: resultado 24;
   - `100 7 5 - /`: resultado 50;
   - `3 +`: mal formada;
   - `2 3 4 +`: mal formada.

Exercícios

8. Na notação pré-fixa, o operador aparece antes de seus operandos. Escreva a função `avaliarPreFixa()` que recebe uma expressão pré-fixa válida, com dígitos de 0 a 9, espaços e os operadores +, −, * e /, e devolve o seu valor. A expressão deve ser percorrida da direita para a esquerda. Exemplos:
   - `- + 5 * 3 2 4`: resultado 7;
   - `/ - 9 3 2`: resultado 3.

Entrega

Exercícios 1 a 3

1. Resolver os exercícios **manualmente**.
2. Escanear ou tirar foto dos exercícios resolvidos.
3. Agrupar as imagens e entregar em um único arquivo `PilhaEncadeadaAplicacoes_NomeSobrenome.pdf`.

Exercícios 4 a 8

1. Organizar cada exercício em uma pasta própria, contendo os arquivos `.h` e `.c` necessários para compilá-lo.
2. Compactar as pastas e entregar em um arquivo `PilhaEncadeadaAplicacoes_NomeSobrenome.zip`.